

NLP Assignment 1

Jana Nikolovska, Marco Sangiorgi, Andrea Fossà

Master's Degree in Artificial Intelligence, University of Bologna
{ jana.nikolovska, marco.sangiorgi24, andrea.fossa }@studio.unibo.it

Abstract

This assignment focuses on the creation of a full NLP pipeline to tackle the [EXIST 2023](#) task on sexism identification in tweets using LSTM and Transformer-based models. Our approach consists of data pre-processing and models creation on which training and evaluations procedure are performed, with the help of the HuggingFace library. We analysed the impacts of different architectural choices and discussed our classification performances, discovering that how we feed our data to the models play a crucial role in the final outcome.

1 Introduction

Binary classification problems can be addressed using several approaches ranging from classical Machine Learning to Deep Learning models.

The former include simple and interpretable Bag-of-Words (BoW) models able to work with word frequencies, especially on small dataset, providing high-speed responses with the help of classical classifiers like SVMs. Their main limitation of lack of word order makes high semantic content difficult to cope with. More sophisticated metric such as TF-IDF can help regulate word importance and achieve better results.

The latter, instead, intrinsically capture complex patterns and relationships, yielding a better semantic understanding. Word embeddings are usually the first block in text-related tasks, mapping words to semantic vectors in a lower-dimensional space that SVMs can easily process. More complex pipelines use RNNs such as LSTM or GRU to capture complex word relationships, though these struggle with long-term dependencies. Transformer-based architectures, such as BERT, focusing on learning good representation of words and their context, dominate many NLP tasks by using attention mechanisms to manage long-term dependencies. However, they are computationally

expensive due to the high time complexity of multi-head attention layers.

That said, we used a simple LSTM model and a pre-trained [Twitter-roBERTa](#) model from the BERT family to address the sexist classification task with and without state-of-the-art Transformer models.

2 System description

For the first part of this assignment, we applied pre-processing steps to our data consisting of text tokenization, lemmatization and encoding. The tokenization step is mainly performed by regex and simple [Pandas](#) Dataframe maps made to delete or replace undesired tweets symbols (e.g. URLs, hashtags, mentions, ...) with special tokens. Specifically, we used [Emoji](#) to replace emojis with their description and [Spacy](#) to replace mentions with their entity type or part of speech. [WordSegment](#) is used to replace hashtags with their own splitted version and [NLTK](#) is used to remove stopwords. Follows the lemmatization step by means of NLTK's Lemmatizers and [AutoCorrect](#) spell checkers. Eventually, the encoding step is performed creating the embedding matrix along with data vocabularies and OOV terms visualizations. We used [GloVe-Twitter](#), fetch via the [Gensim](#) library, as pre-trained embedding model to which OOV training terms were added on static embeddings via nearest neighbour strategy.

For the second part, we used a simple highly-configurable PyTorch LSTM model, flexible enough to fit any custom training loop, (i.e. HuggingFace's one). The architecture comprises an Embedding Layer followed by a LSTM Layer and a Dense Layer, with a Sigmoid activation on top. Created with `lstm_model_create()`, it allows defining the model's `forward()`'s output content along with [BCEWithLogitsLoss](#)'s recall-precision trade-off parameter, and whether or not to apply a weighting of the loss for imbalanced datasets. For training, we used a HuggingFace [Trainer](#), customizable

with desired training-loop’s configurations and metrics, along with custom PyTorch Dataset wrapping the training, validation and test data. [Stratified K-Fold](#) sampling from the Scikit-Learn package is exploited to shuffle datasets’ indices and obtain representative batches to cope with data imbalances.

For the transformer part we used a model specifically trained for hate speech detection, namely Twitter-roBERTa-base. Additionally we fine-tuned it by training it with different seed and a specific train build for weight balancing which showed improvements to the results ¹ (or from the baseline results). Regarding multilanguage analysis with spanish tweets we used the model [XLM-RoBERTa](#). Results in Table 1.

3 Experimental setup and results

For training the LSTM model, we used the PyTorch [AdamW](#) optimizer with 0.5 of weight decay and a [ReduceLROnPlateau](#) LR scheduler. The batches consist of 32 stratified elements and the embedding layer is kept trainable.

In Table 1 are shown the best hyper-parameters searched via a hand-made² GridSearch over number of LSTM layers (*#L*), the sexist output threshold (*Thr.*), the embedding dimension (*Emb*), the *BCEWithLogitsLoss*’s *pos_weight* or recall-precision importance factor (*RPI*), the learning rate (*LR*) and whether or not to apply a weighted loss on the batches (*wL*). The metrics used are F1 scores on validation (*vF1*) and test (*tF1*) sets.

For the transformer instead we used a different batch size equals to 16 and trained for 6 epochs. For the preprocessing (*PRE*) we used both the previous with different embedding dimension (*P*) and the naive one find on the instruction of the RoBERTa model (*S*).

4 Discussion

Our best LSTM models tends to select the higher LR $1E-03$ and overfit on validation set when embedding dim is 200, the reason may lie in a higher number of trainable embeddings. Also, we found that smaller models tend to come up with a higher Recall. In fact, the smaller the model the more weighted loss and higher RPI are used, leading to more positive predictions overall. The best LSTM models on test F1 score fall into no weighted loss and either 0.4 threshold or higher

<i>LSTM Models</i>							
<i>Hyper – parameters</i>						<i>Metrics</i>	
Emb	#L	T	wL	RPI	LR	vF1	tF1
25	1	0.5	T	2.0	1E-3	0.701	0.692
	2	0.4	T	1.0	1E-3	0.702	0.680
50	1	0.6	F	2.0	1E-3	0.727	0.687
	2	0.4	F	2.0	1E-3	0.739	0.698
100	1	0.4	F	1.0	1E-3	0.770	0.718
	2	0.4	F	1.0	1E-3	0.755	0.723
200	1	0.6	F	1.5	1E-3	0.763	0.717
	2	0.5	F	1.0	1E-3	0.786	0.718
<i>Transformers</i>							
Emb	Model			Pre	LR	vF1	tF1
/	RoBERTa			S	1E-5	0.846	0.792
50	RoBERTa			P	1E-5	0.812	0.810
200	RoBERTa			P	1E-5	0.811	0.801
50	XLM			P	1E-5	0.802	0.787

Table 1: F1 scores of our models.

RPI, meaning a tendency of having more output toward 0 and perhaps a response to cope with data imbalances.

The Transformer results are higher than expected, the accuracy and f1 scores are very similar overall showing that data imbalances was solved by using our custom Trainer. We further analyze wrong classified tweets, in particular false negatives allowing us to investigate the reason of the ban received from the annotators. Indeed, our model struggled more with intent categorization (*label-task2*), while on sexist classification (*label-task3*) the model performed quite well especially in identifying content related to sexual violence.

5 Conclusion

Simple LSTM models can lead to decent performances if properly configured, but needs to be specifically tuned. The dimension of the embedding matrix really impacts the overall performance.

Transformer-based models effortlessly outperforms LSTM ones, showing better understanding of context thanks to the introduction of the attention mechanism.

6 Links to external resources

- [GitHub repository](#).

References

¹Inspired by: [Training function](#), [HuggingFace discussion](#).

²[Skorch](#) wrappers for sklearn GridSearch were too slow.